# Tesla Stock Price Analysis and Prediction Report

Python programming for science project


By:

Farah Balobaid -Rawan Alotaibi

Hala Aldossary- Haya Alhajri

Aljawharah Alzuabi- Fatimah Albahrani

Haneen Albalawi

# Abstract

This project presents a comprehensive analysis of Tesla's (TSLA) historical stock data, aiming to uncover market trends and predict future price movements. The study leverages Python's data science ecosystem to transform raw financial data into actionable insights.

The methodology begins with data acquisition and rigorous data wrangling using **Pandas** and **NumPy**. This phase focuses on cleaning the dataset, handling missing values, and performing statistical aggregations to ensure data quality and consistency. Following the preparation phase, we conduct an Exploratory Data Analysis (EDA) to investigate relationships between trading volume and price fluctuations.

To communicate findings effectively, **Matplotlib** is utilized to generate visual representations of the data, highlighting key trends and patterns over time. This report details the end-to-end process—from data cleaning and problem-solving challenges to the final visualization of results—demonstrating how historical data can be used to understand market behavior.

# Table of Contents


| | |
|---|---|
| 4 | Introduction |
| 5 | Data wrangling |
| 6 | NumPy Arrays & Slicing |
| 7-8 | Data Preprocessing with Pandas |
| 9-11 | ML Modeling with Python Libraries |
| 12-16 | Stock Trading Visualizations |
| 17 | Conclusion |
| 18 | Future Work |

# Introduction

**Project Overview & Motivation** The financial market is a complex ecosystem where stock prices are influenced by a multitude of factors. For this project, we selected **Tesla, Inc. (TSLA)** as our subject of study. Tesla is chosen because it represents one of the most volatile and actively traded stocks in the market today. Its price history offers a rich dataset filled with significant fluctuations, making it an ideal candidate for applying data analysis techniques and testing the effectiveness of our predictive models.

**Problem Statement** Investors and analysts face a constant challenge: how to make informed decisions in an unpredictable market. Raw historical data alone is overwhelming and difficult to interpret without proper processing. The core problem this project addresses is the need to transform unstructured historical stock data into clear, visual insights. By answering questions such as "How does trading volume correlate with price changes?" and "What are the long-term trends?", we aim to bridge the gap between raw data and strategic decision-making.

**Project Objectives** The main objectives of this report are:

- To collect and clean historical data for Tesla using Python libraries (**Pandas** and **NumPy**).
- To perform statistical analysis to understand data distribution.
- To visualize market trends and trading volumes using **Matplotlib**.
- To provide a comprehensive summary of the stock's behavior over the selected period.

# Data Wrangling

To prepare the dataset for accurate analysis, we performed a series of cleaning steps using Pandas. We started by addressing data redundancy using the `drop_duplicates()` function. This step was essential because duplicate records can skew statistical results by counting the same data points twice.

Next, to ensure the integrity of our visualizations, we removed any incomplete rows using the `dropna()` method. We decided to remove these rows rather than filling them because our analysis relies on precise historical data, and missing values could lead to errors in the plotting phase. Finally, since removing rows creates gaps in the row numbers, we used `reset_index(drop=True)` to reorganize the dataframe. This ensured that our data had a clean, sequential order, making it ready for the analysis algorithms.

# creating arrays and slicing with Numpy

In the analytical phase of our project, we utilized the **NumPy** library to perform statistical computations. The process began by transforming our data into a format suitable for high-performance calculation. We extracted the key financial columns—specifically Open, High, Low, Close, and Volume—and converted them into NumPy arrays using the `.to_numpy()` method. We specifically chose this approach because NumPy arrays offer significantly faster processing speeds compared to standard lists, which is essential when dealing with large financial datasets. To ensure the data integrity before processing, we printed these arrays to verify their structure and content.

Once the arrays were prepared, we applied statistical functions to derive meaningful insights from the raw numbers. We calculated the **mean** (`np.mean`) for both prices and trading volume to establish a baseline average for the stock's performance over the selected period. More importantly, we calculated the **standard deviation** (`np.std`). In financial analysis, this metric is critical as it serves as a proxy for **volatility**; a higher standard deviation indicates that Tesla's stock price has experienced sharp fluctuations, representing higher risk. Finally, we used the `np.min` and `np.max` functions to identify the lowest and highest trading points, effectively defining the trading range of the stock during our study period.

# filtering , and preparing for graphs,machine learning with pandas library

In this stage, we leveraged the full power of the **Pandas** library to perform deep exploratory data analysis and prepare the dataset for predictive modeling.

**Deep Statistical Analysis (EDA)**

Beyond basic averages, we needed to understand the distribution and integrity of our data. We utilized df.describe() to generate a comprehensive statistical summary (including quartiles), and we specifically calculated the median alongside the mean. We did this because stock prices can have extreme outliers; comparing the median to the mean helps us identify skewed data. Furthermore, we examined the Standard Deviation (std) to quantify market volatility. To ensure our data types were correct for calculation, we used df.dtypes and df.info(), and we ran a frequency analysis using value_counts() on the 'Volume' column to see how often certain trading volumes occurred.

**Data Integrity Checks**

Before moving forward, we performed rigorous quality checks. We used df.isnull().sum() to scan every single column for missing values, ensuring our dataset was complete. We also validated the 'Date' column by removing any rows with invalid dates using dropna(subset=['Date']), as accurate timelines are non-negotiable for time-series forecasting.

**Preparing for Machine Learning (Feature Engineering)**

Finally, we structured the data for the Machine Learning phase. We split our dataset into two distinct parts:

1. **Features ($X$):** We selected `['Open', 'High', 'Low', 'Volume']` as our input variables.
2. Target ($y$): We selected ['Close'] as the value we want to predict.

This separation is the foundational step for training any AI model, where the algorithm learns patterns from the "Features" to predict the future "Target" price.

# Machine Learning Modeling with SCIPY , TensorFlow and Scikit-learn

In this core phase of the project, we implemented three distinct modeling techniques to predict Tesla's stock closing price.

### 3.1 Baseline Modeling with Scikit-Learn

We began by establishing a baseline using the **Linear Regression** algorithm from Scikit-Learn. We fitted the model to our features (Open, High, Low, Volume) to predict the Closing Price. To validate our results visually, we plotted the predicted values against the actual values. As shown below, the overlapping lines confirm that the linear model effectively tracks the general market trend.

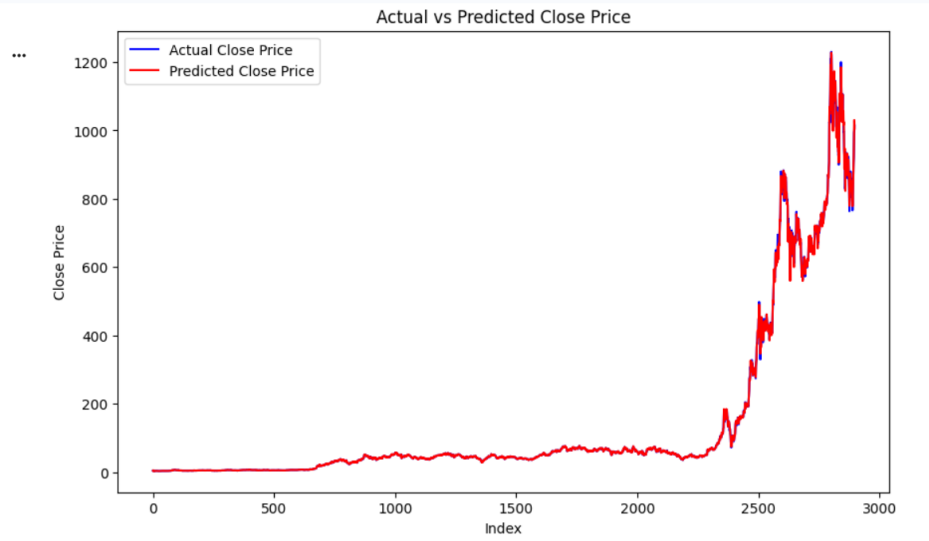



*Figure 1: Comparison between Actual and Predicted Closing Prices using Scikit-Learn.*

### 3.2 Mathematical Optimization with SciPy

Next, we utilized **SciPy** to demonstrate the mathematical optimization behind the scenes. We defined a Cost Function and used the **BFGS algorithm** to minimize the error iteratively. The chart below illustrates how the regression line fits the data points after optimization.

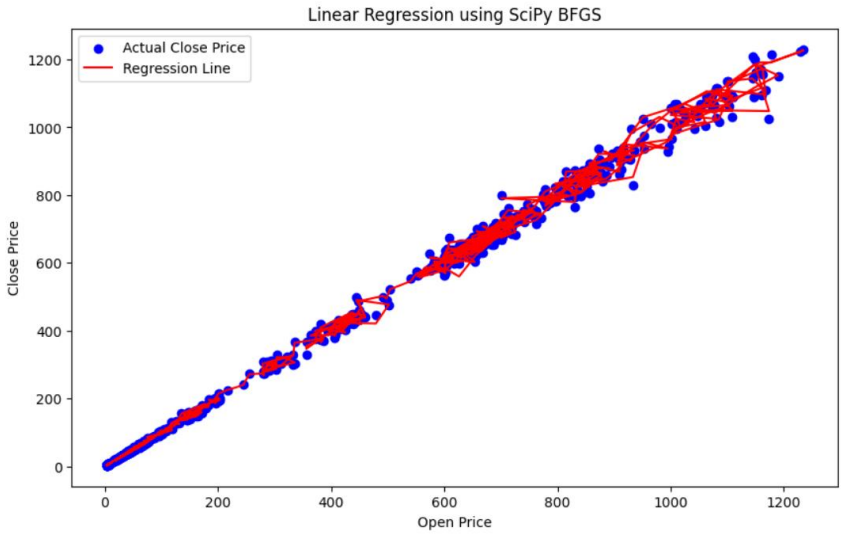



*Figure 2: Linear Regression fit optimized using SciPy's BFGS algorithm.*

### 3.3 Deep Learning with TensorFlow

Finally, we advanced to Deep Learning using a **Sequential Neural Network** in TensorFlow. We trained the model over 50 epochs. The graphs below display the learning process (Loss & MAE), showing a steady decrease in error, which indicates successful training.

...

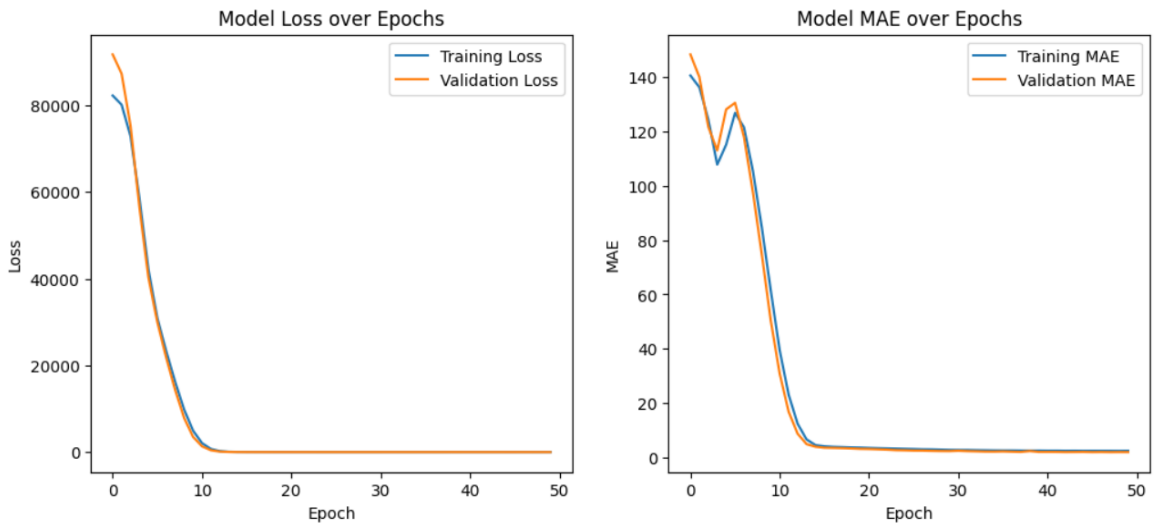



*Figure 3: Model Loss and Mean Absolute Error (MAE) over 50 Epochs.*

After training, we tested the model on unseen data. The scatter plot below demonstrates the high precision of our Neural Network predictions.

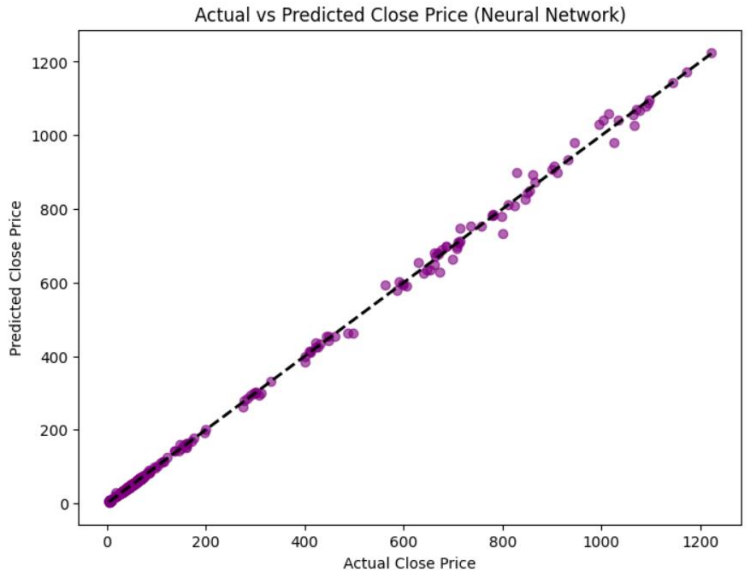
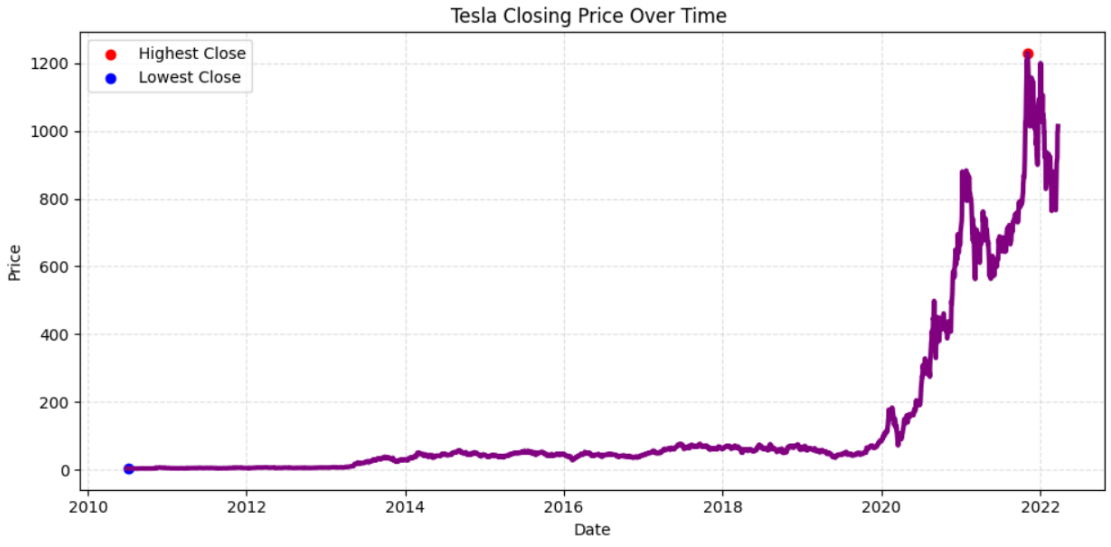



*Figure 4: Actual vs. Predicted Prices using the TensorFlow Neural Network.*

# visuals for stocks trading with matplot

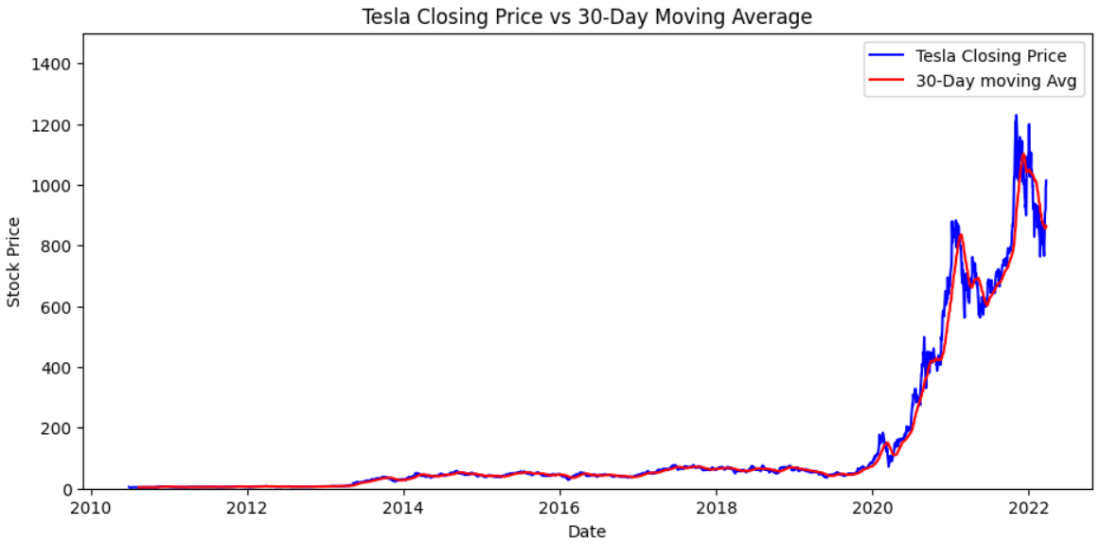
In this section, we used the Matplotlib and Pandas libraries to create a wide variety of statistical plots that help traders better understand Tesla's stock behavior. We began by loading and preparing the dataset using `read_csv`, converting the Date column into a datetime format, and sorting the data chronologically to ensure accurate visual analysis.

Our visualization process started with a **line plot** showing Tesla's closing price over time. We highlighted the highest and lowest closing values using scatter points, giving a clear view of major peaks and dips in the stock's performance.

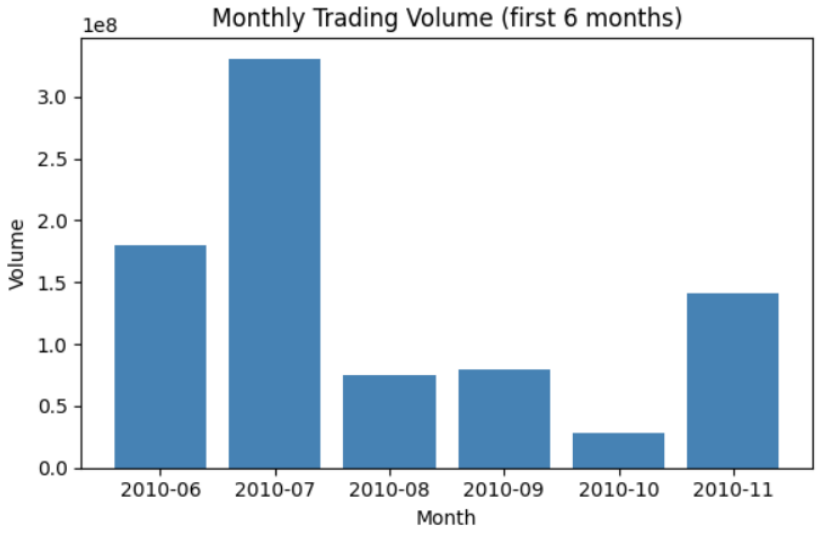



Next, we added a **30-day moving average** to smooth out short-term fluctuations and reveal the long-term trend. This comparison

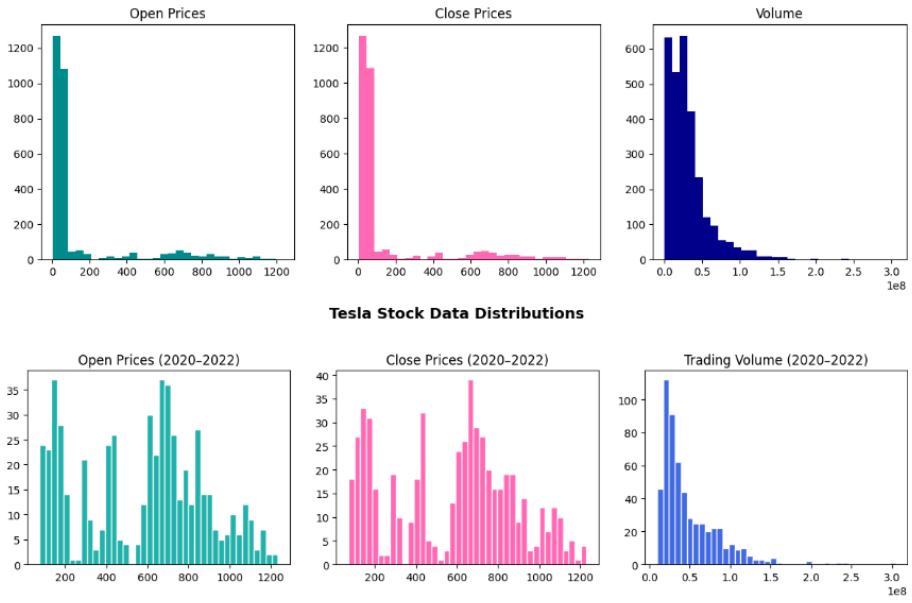
between daily prices and the moving average helps traders identify momentum changes more easily.




To analyze market activity, we generated a **monthly trading-volume bar chart** for the first six months in the dataset. This allowed us to observe which months had unusually high trading activity.



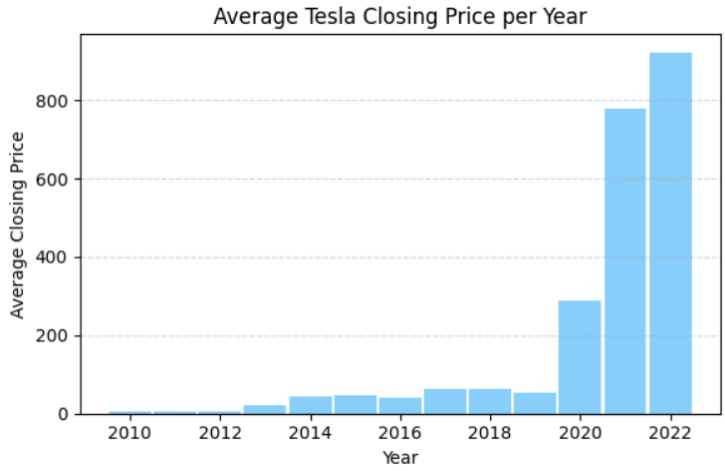
We then created multiple **histograms** for the Open, Close, and Volume columns to study the distribution of values and detect any outliers. Additional histograms focusing on the years **2020 to 2022** helped narrow down patterns during a more recent and volatile period.




To understand seasonal market behavior, we used a **pie chart** to display the top six months with the highest trading volume. This visual representation highlights which months experienced the strongest market interest.

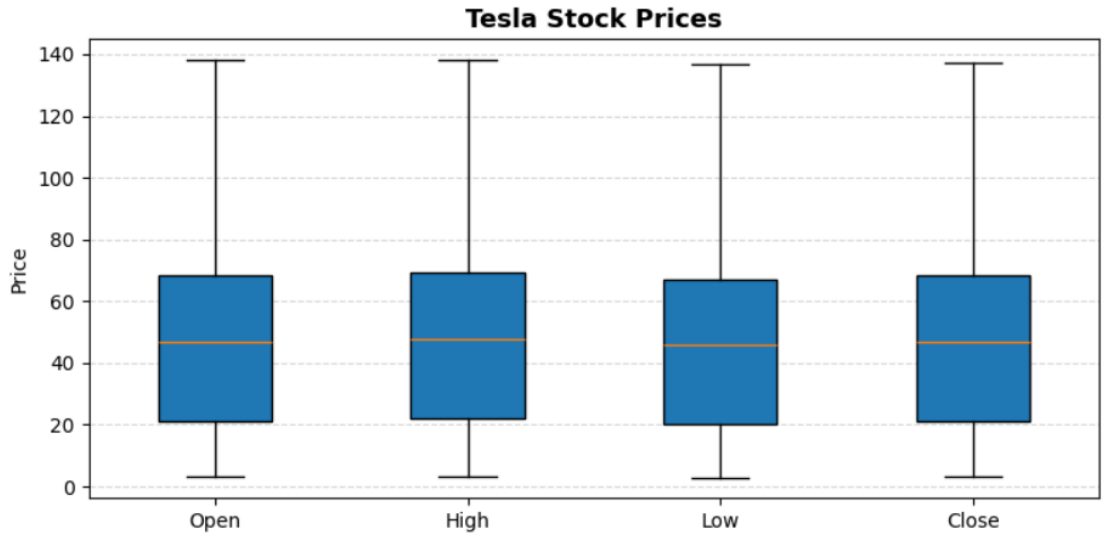



We also plotted a **bar chart** showing the average closing price per year, giving a clean overview of how Tesla's annual performance has evolved over time.

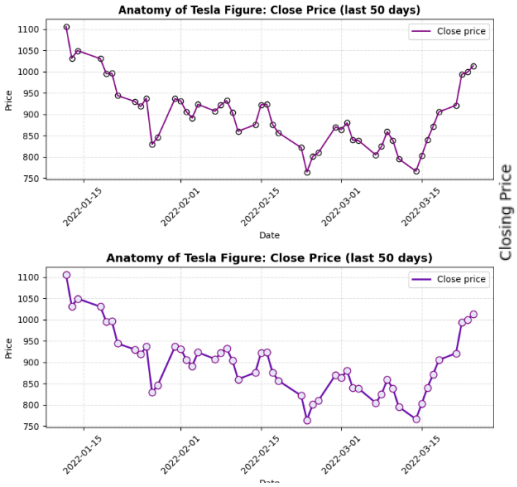



Next, we used a **boxplot** to summarize the range and distribution of the main price columns (Open, High, Low, Close). This provided

insight into price variability and helped identify typical value ranges.




Finally, we visualized Tesla's **most recent 50 trading days** using a line-and-scatter combination to highlight short-term trends. We also created a **scatter plot comparing trading volume to closing price** from 2020 to 2022, which showed that stronger price movements often occurred during periods of higher trading activity.






# Conclusion

This project successfully demonstrates how Python can be used to transform raw historical stock data into meaningful insights. Using Pandas, NumPy, and Matplotlib, we conducted a thorough statistical and visual analysis of Tesla's stock performance.
Machine learning models—including Scikit-learn Linear Regression, SciPy optimization, and a TensorFlow Neural Network—were implemented to predict closing prices.

**Main Takeaway:**
The TensorFlow Neural Network outperformed traditional models due to its ability to capture nonlinear patterns and market volatility.

Overall, the project achieved its objectives, providing a complete workflow from data cleaning to predictive modeling and visualization.

# Future Work

There are several ways this project can be improved in future work:

**1. Add More Features**

Include additional technical indicators such as moving averages, RSI, or MACD to help the models capture market trends more accurately.

**2. Use Advanced Models**

Apply time-series oriented models like LSTM or GRU, which are better at learning long-term patterns in stock data.

**3. Expand the Dataset**

Incorporate more years of data or external factors such as financial news, trading sentiment, or economic indicators.

**4. Improve Model Tuning**

Experiment with hyperparameter tuning to improve model accuracy and reduce prediction errors.